

CUDA PROGRAMMING FOR NVIDIA H100S

WGMMMA Part 2



Prateek Shukla · 简体中文版

对 **wgmma** 操作分组

WGMMA 发射是异步的。wgmma.mma_async 把工作放入"在途"状态并立即返回。

如果我们必须逐个"跟踪每一条 MMA"，我们要么付出沉重的 scoreboard 开销，要么被迫使用过度保守的"等待一切"屏障。

组给了我们流水线友好的粒度。

wgmma.commit_group = "关闭当前批次"（让它可跟踪）

wgmma.wait_group N = "只有在太多批次在途时才停顿"

这实现了重叠：当组 g 在计算时，你可以为组 g+1 做准备/加载（地址计算、TMA、staging 等）。

wgmma.commit_group.sync.aligned;

这只是一种把所有尚未提交的 wgmma.mma_async 操作捆绑起来的方式

硬件可以同时跟踪多个组。通过分组，你降低了逐条跟踪每次矩阵乘法的开销

你通常发射一组 WGMMA 指令来计算输出的一个"tile"（例如一个 64x64 的块），然后把该 tile 作为一个组提交。

然后我们对这些操作进行等待

wgmma.wait_group.sync.aligned N

这是同步点。它说："暂停这个线程，直到只剩 N 个已提交的组仍在运行。"

`wgmma.wait_group.sync.aligned 0` - 等待一切完成。当你即将读取寄存器中的最终结果并把它写入全局内存 (global memory) 时，就用它。

`wgmma.wait_group.sync.aligned N` - 让 N 个组在后台继续运行，同时你继续为下一个组准备数据

"提交/等待"时间线（实际重叠的是什么）

你不是简单地"提交然后等待"。你把它流水线化。

发射一串 `wgmma.mma_async`（这是一个输出 tile）`wgmma.commit_group.sync.aligned`;

开始准备下一个 tile（TMA、指针计算等）

`wgmma.wait_group.sync.aligned N`;（让 N 个组保持运行）

排空

`wgmma.wait_group.sync.aligned 0`;

现在可以安全地读取累加器 D 并把它存出去了

wgmma.fence.sync.aligned

面向 WGMMA 流水线的 warpgroup 栅栏。它承担两个紧密相关的角色：

WGMMA 发射的 arrive/组边界标记：WGMMA 操作按组发射。该栅栏用来确立一个组的发射窗口的起点。

对 WGMMA 所用操作数的寄存器排序/冒险控制

wgmma.mma_async 是异步的，使用 warpgroup 级的执行流水线。该栅栏用于防止涉及累加器寄存器以及任何驻留寄存器的操作数碎片（尤其是当 A 以寄存器为来源时）的重排序/冒险。

它不是跨 proxy 栅栏。如果你要读取经寄存器修改过的 TMA 数据，你需要一个跨 proxy 栅栏：fence.proxy.async（或等价的 acquire/release 发布协议）。

store

当 WGMMA 指令完成时，你的结果矩阵 C 正躺在累加器寄存器里。结果并不是以连续矩阵的形式存储的（例如第 0 行、第 1 行、第 2 行.....）。相反，它以高度不透明的方式碎片化分布在 128 个线程的寄存器中

在普通 store (st.shared) 中，如果 Thread 1 写数据，就由 Thread 1 提供地址；我们不能用这种方式，因为这将需要手工管理地址。

在 stmatrix 中，数据提供者与地址提供者是解耦的，就像 ldmatrix 一样。

stmatrix 指令

每个线程输入它在自己的寄存器中从 wgmma 得到的数据，然后给出要把这些数据写入其中的目标指针

.m8n8：原子单元仍然是 8x8 tile。

.x4：每线程写 4 个寄存器（合计 16x16 tile）。

.x1、.x2：也支持更小的 tile。

.shared：目标总是共享内存。

stmatrix.sync.aligned - 我们在为一个矩阵存储数据，所有 **warp** 必须

在继续之前同步，并保证每一个线程都在发射该指令

stmatrix 与 fp32

最需要理解的关键点是：stmatrix 不支持 f32 数据。它只支持 .b16（打包 16 位）和 .b8（打包 8 位）类型。这在如何处理 wgmma 输出的问题上制造了一个分叉。

使用 FP32 累加器（.f32）时，寄存器状态：累加器驻留在未打包的 .f32 寄存器中。你不能把这些寄存器喂给 stmatrix。你必须先向下转换并打包它们（是的，这会给你有损的结果）

如果使用 FP16 累加器（.f16），累加器驻留在打包的 .b32 寄存器中（被视为 .f16x2）。一个 32 位寄存器容纳两个元素，因此不需要转换和打包

stmatrix

与 ldmatrix 一样，stmatrix 是协作式的。warp 中的每个线程（0 到 31）都提供自己的指针。每个线程给出其数据所属的共享内存地址。

由于 warp 中的线程以一种非常特定、非线性的模式持有数据，它们生成的地址也必须同样非线性，以确保数据以连续、干净的矩阵形式落入共享内存

当你执行 stmatrix 时，该指令从 [addr] 开始，把这些元素以行主序（连续行）写入 SMEM。

m8n8 tile 结构

该指令作用于一个由 16 位元素组成的 8x8 tile。

线程 0-31 : warp 协作。

x1 变体 : 每个线程持有一个 8x8 矩阵的一部分。

x4 变体 : 每个线程持有四个 8x8 矩阵的数据

与 Idmatrix 一样，它假设特定的线程持有特定的行。线程 0-7 持有第 0 行，依此类推。

阶段 **1**：地址责任（谁指向哪里？）

这的工作方式与 Idmatrix 完全相同。线程 0-7：指向第 0-7 行（左半部分，第 0-7 列）。

线程 8-15：指向第 8-15 行（左半部分，第 8-15 列）。

线程 16-23：指向第 0-7 行（右半部分，第 0-7 列）。

线程 24-31：指向第 8-15 行（右半部分，第 8-15 列）。

寄存器来源（谁持有什么）

"条纹式"布局：线程 0 充当整个 tile 的"第 0 与第 1 列"所有者。它看到的不是连续的行，而是垂直切片。

线程 0 寄存器映射：寄存器 r0：矩阵 A（左上）-> 第 0 行，列 [0, 1]

寄存器 r1：矩阵 B（左下）-> 第 8 行，列 [0, 1]

寄存器 r2：矩阵 C（右上）-> 第 0 行，列 [8, 9]

寄存器 r3：矩阵 D（右下）-> 第 8 行，列 [8, 9]

序列化循环

由于 `stmatrix.x4` 每次调用每个线程只消耗 4 个寄存器，而一个输出 tile 的累加器更大。所以我们对累加器的"切片"做循环：

切片 i -> 打包进 $\{r0, r1, r2, r3\}$

`stmatrix(...)`

切片 $i+1$ -> 打包 -> `stmatrix(...)`

这就是为什么 epilogue 通常是一个循环，而不是单次 store。

核心问题

我们做 swizzle 的"原子"是 16B。

16B 很重要，因为 m8n8 bf16 tile 的单行是 8 个元素 = 16 字节。所以每一行天然就是一个 16B 块。

诀窍在于：我们 swizzle 的是行原子，而不是单个元素

即使原子被重排了，每个 16B 原子内部的数据仍保持行主序

这就是为什么 TMA 能"unswizzle"回一个干净的行主序矩阵。

FP8

FP8 张量核心在理论上提供 BF16/FP16 两倍的 TFLOPs，在 H100 上达到 ~3958 TFLOPs（所示数字包含稀疏性。稠密约为这些值的 ~12）。这来自更低的精度

位宽减半降低了全局内存带宽压力，并允许在 L2 中缓存 2 倍大的模型/批次。

大部分收益在 matmul 密集的部分（线性层、attention 投影、MLP）。许多流水线仍把一些算子保留在 BF16/FP16（类似 softmax 的操作、归一化、古怪的归约）。

范式转变

直到大约 2022 年，AI 的标准是 FP32 或 FP16/BF16。FP8 主要是一项由 Nvidia 的 H100 (Hopper) 架构驱动的工业进步，它让研究人员能用一半的内存、以两倍的速度训练模型。

很多真正的大模型，比如 deepseek V3，使用 fp8 进行训练；而 kimi k2 则以使用 int4 量化感知训练 (Quantization-Aware Training) 而闻名。

与 INT8 不同，FP8 是一种“浮点数”，意味着它能表示很宽的动态范围。由于 8 位非常局促，工程师把 FP8 拆成两种专门化的格式来平衡精度与范围：e4m3 和 e5m2

低精度下的 **Wgmma**

我们有 2 种更低精度

- e4m3 -
 - 4 位指数和 3 位尾数
 - 范围：448（最大正规值 448，最小正规值 0.015625）
 - 主要用于权重和激活值（前向传播），因为它有良好的动态范围并能表示 NaN/Inf。
- e5m2 -
 - 5 位指数和 2 位尾数
 - 范围：57344（远大于 E4M3）
 - 主要用于训练梯度和反向传播，因为它能表示梯度更新期间所需的非常大的数值。

饱和

在 FP32 中，你的动态范围是巨大的（ $\sim 10^{38}$ ）。你实际上永远碰不到天花板

在 FP8 E4M3 中，“天花板”是 448。如果你的矩阵乘法结果出现了一个 450 这样的值，而你没有处理它，你就撞上了饱和。当你把 float 向下转换成 FP8 时，硬件必须决定如何处置那个 450。它通常有两种模式（由 intrinsics 控制）

值被钳制到最大可表示数（448）。这会扭曲你的数据（clipping），但让数学保持“稳定”。

值变成无穷大（Infinity）（在 E4M3 中是 NaN，因为它没有 Inf 表示）。这会立刻毁掉你的训练运行

缩放因子

既然改变不了 8 位的物理现实，我们就只能作弊。我们使用缩放因子（Scaling Factors）。这是 H100 上 FP8 的核心课程内容。

张量级缩放（Tensor-wise Scaling）：你为整个张量矩阵计算单一的缩放因子（最大值）。这实现起来最简单，开销最低

向量级缩放（Vector-wise Scaling）：你为每一行或每一列分配一个独有的缩放因子。

块级缩放（Block-wise Scaling，MXFP8 / Micro-scaling）：矩阵被切成小 tile（例如 16x16 或 32x32）。每个 tile 有自己的缩放。

还有其他的，比如 SmoothQuant、Delayed Scaling 等

打包

FP8 在传输期间并不是以独立对象存在的。H100 内存控制器不会为任何小于 32 字节（一个"sector"）的请求而启动。如果你请求 1 字节（一个 FP8 值），GPU 会取回 32 字节，把你所要的那 1 字节给你，然后把其余 31 字节扔掉。

为了修复这种浪费，我们有了打包类型 `e4m3x2/4` 和 `e5m2x2/4`。它们是硬件能理解的、彼此独立的 PTX 数据类型。

x4 打包（寄存器填充器）类型：`.e4m3x4` 或 `.e5m2x4`

x2 打包（数学输入）类型：`.e4m3x2` 或 `.e5m2x2`

x4 打包

这些是包含 4 个 fp8 值的打包数据类型：e4m3x4 和 e5m2x4

总大小：8 位 $\times$ 4=32 位。

它为什么存在：它恰好完美填满一个标准 GPU 寄存器。

布局（小端）：

[元素 3|元素 2|元素 1|元素 0]

<-- MSB (31) LSB (0) -->

这是你用于存储和传输的主要格式。当你在移动数据或做简单数学（比如求最大值）时，你要的是 x4。

x2 打包

包含 2 个 fp8 元素的打包数据类型：`.e4m3x2` 或 `.e5m2x2`

总大小： $8 \text{ 位} \times 2 = 16 \text{ 位}$ 。

这与 FP16 (half) 的大小匹配。

H100 的设计就是为了让人们从 FP16 过渡到 FP8。

硬件有专用逻辑，可以把一个装着两个 FP16 的 32 位寄存器直接压缩成一个装着两个 FP8 的 16 位寄存器。

张量核心通常以这些 16 位块来摄取数据。

Fp8 转换/量化

如果你的源数据是 FP16/FP32，你会通过 `cvt` 把它转换成 FP8（通常用 `.e4m3x2/.e5m2x2` 这样的向量形式），并且通常带 `.satfinite`，让超出范围的值被钳制：

在喂给 WGMMA 之前，先决定好你的量化策略（舍入模式、是否 `satfinite`、任何 `per-tensor/per-channel` 缩放）。

PTX 描述了饱和行为：如果 `|input|` 超过目标格式的最大正规值，结果就变成保留符号的最大正规值。

PTX 为 `sm_90` 或更高架构提供了 `cvt.satfinite.{e4m3x2,e5m2x2}.{f32,f16x2}` 支持

``cvt.rn.satfinite.e4m3x2.f16x2`` 这条指令取 2 个 fp16 值，对它们做饱和处理、舍入，然后打包

搭载 **fp8** 的 Hopper 第 4 代张量核心

输入是 8 位 (FP8)，但为了数值稳定性，累加发生在 32 位 (FP32) 寄存器中。

通过 WGMMMA 原生支持 E4M3 和 E5M2 两种输入

一条 WGMMMA 指令发射一大块数学运算，有效隐藏延迟

虽然寄存器是 FP32，但它的表现像 FP22((~8 位指数 + ~13 位尾数)，一个已知的硬件特性。

我们本质上是把 FP8 当作压缩存储格式，同时在"伪 FP32"空间中做数学。

FP8 指令语法

`m64nNk32` : 固定的 $M=64$ 、 $K=32$ (对 8 位而言)。N 是可变的 (例如 8、16、.....256)。

`.f32` : 累加器 D 的类型 (单精度)。

`.e4m3.e4m3` : 矩阵 A 和 B 的输入类型。FP8 的特殊之处在于 A 和 B 可以是不同的 FP8 格式 (例如 `.e4m3 x .e5m2`)。PTX 示例展示了混用的 `.e4m3/.e5m2` 配对。

`scale_*` : 这些是缩放因子, 是那些可以在 -1 到 1 之间取值的符号翻转器 (A 和 B 取 $\{-1, 1\}$, `scale_d` 取 $\{0, 1\}$)。

$K = 32$ 是关键差异, 这意味着我们一次性消费 A 的 32 列

A 放在寄存器上

对于 FP8 RS WGMMA，PTX 把 A 操作数表示为直接传给 WGMMA 的寄存器向量表达式。

"显而易见"的做法（用 `ldmatrix ... .b8` 加载真正的 8 位元素）在 SM90a 上不是选项。PTX 说明指出，`ldmatrix` 带 `.b8` 在 `sm_100a` / 更晚架构上才受支持，Hopper 不支持。你不能把 `ldmatrix` 用于 `fp8`

这就是为什么大多数 Hopper FP8 WGMMA 实现都采用以下两种做法之一：

SS 路径：对 A 和 B 使用描述符，或者

RS 路径：通过普通共享内存加载（`ld.shared.b32` / 向量化）把 A 装进寄存器，并已按 WGMMA 想要的打包方式排布好，而不是通过 `ldmatrix`。

严格的 **K-major** 规则

与 FP16/BF16 变体不同，`wgmma.mma_async` 不暴露转置控制（`imm-trans-a/b`）没有转置，WGMMA 就按默认的 K-major 规范布局来解释共享内存操作数。如果你喂的是 MN-major tile，指令无法“重新解释”它们，所以你反正都得打包/转置那个转置/打包步骤会增加额外的指令、同步和共享内存流量，损害吞吐量。MN-major 还让 FP8 的 swizzle 原子对齐/整除约束变得复杂（128b 原子缩放）以 K-major 暂存 FP8 tile（或离线预打包）；只有当你会在 staging 期间转置时才使用 MN-major。

精度陷阱

即使 wgmma 指令在文档中写明使用 FP32 累加器，实证研究也报告过：FP8 张量核心累加的行为可能像一种降低精度的 FP32 变体（ ~ 8 位指数 + ~ 13 位尾数，即"FP22 总位数"），这意味着最低的那些 FP32 尾数位可能在累加过程中实际上丢失。

对于非常大的点积（大 K），这种缩减的有效尾数会增大舍入/截断误差，而且误差会随归约深度叠加。

使用 K 切分和/或多阶段累加（先累加出部分和，再以更高精度归约各部分），来限制每个 WGMMA"块"的累加深度，并改善数值稳定性。

CUDA PROGRAMMING FOR NVIDIA H100S

FP8 wgmma 指令

Prateek Shukla · 简体中文版

稀疏 WGMMA

这是一种特殊类型的 WGMMA 操作：张量核心做同样的 MMA 数学，不同之处在于 A 是结构化稀疏的（50% 零值）。

descA（或 A 的寄存器碎片，取决于 RS 还是 SS 形式）、descB（对应 B）sp-meta（一个包含打包索引的 .b32 寄存器），sp-sel（一个 32b 常量，即"稀疏选择器"，选择哪些线程为一个组贡献元数据）以及其他操作数.....

一些要点

硬件严格按稀疏 $A \times$ 稠密 B 运作。即使矩阵 B 在数学上是稀疏的（含零值），张量核心也把它当作稠密矩阵对待。

我们需要为每一个线程创建一个名为 `sp_meta` 的寄存器。你只需让 `sp_meta` 在硬件会读取的 lane 上有意义；在其他 lane 上，你通常把 `sp_meta` 设为 0（或任意值），因为（对这些形状）那些 lane 的元数据会被忽略。

打包与元数据

NVIDIA 稀疏张量核心的物理规则是 2:4 结构化稀疏。我们提供一个由 4 个值组成的连续向量，叫做 Quartet（四元组）（我可没编造这个词）。你必须恰好删掉其中 2 个，并存储两个幸存者（是的，这必须发生在全局内存中，或者在共享内存里经过某种变换）

这节省了 50% 的带宽和存储。如果你把 [8.5, 3.2] 交给 GPU，它完全不知道它们属于哪里。8.5 来自索引 0、1 还是 2？这就是元数据登场的地方。

既然我们要从 4 个可能的位置（0、1、2、3）中选出 2 个索引，我们就需要编码幸存者的位置。

sp-sel

spSel 告诉硬件谁负责提供元数据

换句话说：在每 4 个连续线程组成的组（T0-T3）里，哪个线程对被视为元数据贡献者——适用于只有一对线程做贡献的情形。

TF32 稀疏（.m64nNk16 .tf32）：spSel 必须是 0（线程 T0,T1）或 1（线程 T2,T3）。

FP16/BF16 稀疏（.m64nNk32 .f16/.bf16）：spSel 必须是 0（T0,T1）或 1（T2,T3）。

FP8 / INT8 稀疏（.m64nNk64 .e4m3/.e5m2/.s8/.u8）：

所有线程都贡献元数据，所以 spSel 必须是 0（任何其他值都是未定义的）。

配置 **sp-sel** (线程选择器)

选择 0 告诉硬件从线程 T0 和 T1 读取元数据寄存器，忽略 T2 和 T3。

选择 1 告诉硬件从线程 T2 和 T3 读取元数据寄存器，忽略 T0 和 T1。

选择规则 1 (复制式)：如果你的加载器把元数据广播给所有线程 (常见情况)，为简单起见总是选择 `sp-sel=0`。

选择规则 2 (分片式)：如果你为了省寄存器而拆分元数据，你必须动态地让 `sp-sel` 匹配持有数据的线程。

错误的配置会导致张量核心从空寄存器读取，把该块当作稠密或全零来处理。

sp-meta

它告诉硬件 A 里的非零值在哪里

spMeta 是一个打包的位域 (bitfield)。确切映射见 PTX 图示，但规则是：

2:4 结构化稀疏 (FP16/BF16、FP8、INT8)：A 在每行每 4 个相邻元素中有 2 个非零值。只存储这两个非零值，它们的位置 (0..3) 由元数据操作数中的两个 2 位索引编码。

1:2 结构化稀疏 (TF32)：A 在每 2 个相邻元素中有 1 个非零值。元数据使用一个 4 位索引来指示 2 个位置中哪一个存在，且只有两种特定位模式是有意义的；其他值都是未定义的。

为 **FP16/BF16/INT8** 配置 **sp-meta**

对标准精度而言，一次寄存器加载打包出的元数据恰好够四条连续的 WGMMA 数学指令使用。

选择规则：你必须把主循环展开 4 次，并让 sp-meta 在索引 0、1、2、3 之间轮转。

第一个 K-tile 用 sp-meta=0，第二个用 sp-meta=1，依此类推，直到缓冲区被耗尽。

这个索引会把硬件内部指针递增到已加载寄存器内的下一组 2 位索引。

若不递增这个值，硬件将被迫在所有计算中复用第一个 tile 的稀疏模式。

为 **TF32** 配置 **sp-meta** (例外情况)

TF32 元数据需要一种独特的选择策略，因为寄存器只装得下两条 WGMMA 指令所需的信息。你严格不能使用顺序索引 (0、1)；你必须在硬编码的硬件位掩码 14 和 4 之间切换。第 1 步：对于第一次 K=16 计算，你必须使用 $\text{sp-meta} = 0b1110$ (14) 来解码低位。第 2 步：对于第二次 K=16 计算，你必须使用 $\text{sp-meta} = 0b0100$ (4) 来解码高位。需要这些特定值，是为了让硬件 swizzler 与非标准的 19 位/32 位 TF32 数据格式对齐。对 TF32 使用 0 或 1 这样的标准索引，会导致硬件错位和错误的矩阵输出。

CUDA PROGRAMMING FOR NVIDIA H100S

sp-sel 与 sp-meta 的有效取值

Prateek Shukla · 简体中文版

CUDA PROGRAMMING FOR NVIDIA H100S

稀疏 **wgmma** 的不同版本

Prateek Shukla · 简体中文版